

and proper tracking, it becomes challenging to maintain an audit trail of changes made to the Kubernetes infrastructure.

Limited collaboration: In team environments, collaboration between developers and operations personnel can be challenging when managing Kubernetes configurations manually. Communication gaps and misunderstandings may occur.

Complex rollbacks: Rolling back changes to a previous stable state can be a complex and manual process, potentially leading to downtime and service disruptions.

Inconsistent environments: Manually managing configurations for different environments (e.g., development, staging, production) can result in inconsistencies, leading to unexpected behaviour and making it harder to test and reproduce issues.

Security vulnerabilities: Manually applying changes to Kubernetes may increase the risk of security vulnerabilities, especially if there are deviations from best practices or if proper access controls are not enforced.

DevOps bottlenecks: Without automation, development teams might rely heavily on operations personnel to manage deployments and configurations, leading to DevOps bottlenecks and slower delivery cycles.

Scalability challenges: As Kubernetes clusters and applications grow in complexity, manual management becomes less scalable and may hinder efficient scaling and maintenance of the infrastructure.

Let us look at how to mitigate these challenges and promote streamlined, collaborative, and secure operational workflow that aligns well with modern DevOps practices and containerised environments.

GitOps

GitOps is a modern software development and operations approach that uses version control systems, typically Git, as the single

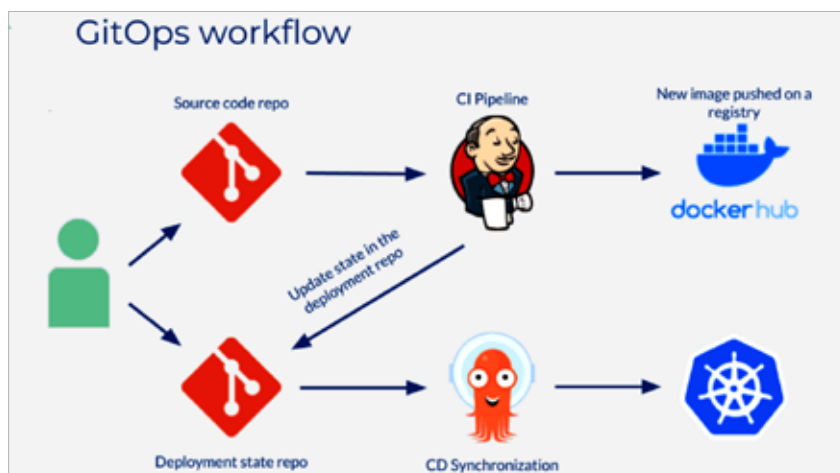


Figure 1: GitOps

source of truth for managing infrastructure and application configurations. The core idea behind GitOps is to treat the entire system as code and use Git workflows to automate the deployment and management of applications in a containerised environment, such as Kubernetes.

Key principles of GitOps

Here we explore how GitOps leverages version control for automated and declarative management, bringing simplicity and efficiency to modern software workflows.

Declarative infrastructure as code: In GitOps, all aspects of infrastructure and application configurations are defined declaratively in code, specifying the desired state of your system rather than writing imperative commands to achieve that state.

Git as source of truth: Git is used as the central repository to store the desired state of the system. Any changes to the infrastructure or application configurations must be made through commits to the Git repository.

Continuous deployment and automation: GitOps workflow emphasises continuous deployment and automation. Changes made to the Git repository trigger automated processes and apply those changes to the target environment.

Reconciliation loop: GitOps employs a reconciliation loop to ensure that the actual state of the system matches the desired state defined in the Git repository. If any deviations are detected, the system automatically makes corrections to bring itself back to the desired state.

How GitOps works

The GitOps workflow typically involves these steps for seamless orchestration.

- Desired state in Git:** Developers define the desired state of the infrastructure and applications in Git repositories as code. This includes Kubernetes manifest files, application configurations, and other related resources.
- Git repository as source of truth:** The Git repository becomes the source of truth for the entire system. Any changes to the system should be made through Git commits.
- Continuous deployment:** A GitOps operator or tool (e.g., Flux, Argo CD) watches the Git repository for changes. When changes are detected, the operator applies those changes to the Kubernetes cluster automatically.
- Reconciliation loop:** The GitOps operator continuously reconciles the actual state of the cluster with the desired state defined in the Git repository. If any drift is